

Pi Extension

高级实践与推荐指南

从 Permission / Sandbox / Dynamic Tools 到 Subagent / Workflow / Compaction

研究版 · 2026-08-14

核心判断

Pi Extension 的价值不在“插件数量”，而在于它把 Tool、Context、Session、Policy、Execution、Subagent 等 Agent Runtime 能力暴露成可组合的 Extension primitives。

目录

1. Pi Extension 的正确心智模型
 2. 推荐生态总览与优先级
 3. 官方 Extension：最值得源码级拆解的 7 类
 4. 社区 Extension：适合日常 Coding Agent 的高价值组件
 5. 五个最值得深入研究的 Extension
 6. 推荐组合：日常开发 / 高级 Agent / Agent Runtime 研究
 7. 源码阅读顺序与学习路径
 8. 从 Extension 反推 Agent 架构原则
 9. 安装与安全注意事项
- 附录 A：Extension 选型速查表
- 附录 B：参考资料

1. Pi Extension 的正确心智模型

很多人第一次接触 Pi Extension，会把它理解成“给 CLI 加命令”。这个理解太窄。更准确的定位是：Extension 是运行在 Agent Harness 中的 Runtime Middleware。

```

User
↓
Input / Session
↓
before_agent_start
↓
System Prompt / Context
↓
LLM
↓
Tool Call
↓
tool_call hooks
↓
Policy / Permission / Routing / Sandbox
↓
Tool Execution
↓
tool_result hooks
↓
LLM

```

因此一个 Extension 可以承担完全不同的职责：

- Tool Extension：注册新 Tool，或者覆盖内置 Tool。
- Policy Extension：在 Tool 执行前进行 allow / ask / deny。
- Context Extension：动态修改 System Prompt 或模型 Context。
- Memory Extension：接管 Compaction，保存和恢复长期状态。
- Execution Extension：把 bash/read/edit/write 路由到 Sandbox、Docker、SSH 或 VM。
- Workflow Extension：实现 Plan Mode、阶段式工作流、Ralph Loop、Subagent 编排。
- UI / Observability Extension：展示状态、审批、日志和 Tool 审计。

Know-why

好的 Agent Core 不应该不断硬编码 Plan、Permission、Memory、Subagent、Sandbox。Core 应提供稳定 lifecycle 与 primitives，把 workflow feature 放到可组合的 Extension 层。

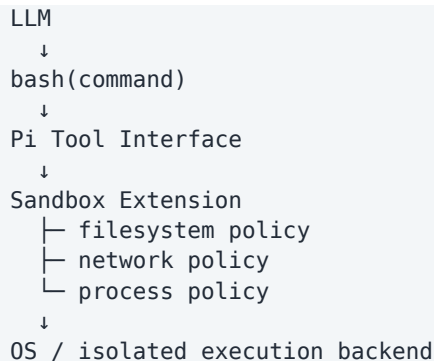
2. 推荐生态总览与优先级

Extension	来源	推荐	主要用途	核心 Know-how
Sandbox	官方	★★★★★	安全执行 / Tool Override	Execution Backend + Capability Boundary
Subagent	官方	★★★★★	独立 Agent 并行/串行	Context Isolation + Orchestration
Custom Compaction	官方	★★★★★	上下文压缩/长期任务	Agent Memory + Context Engineering
Dynamic Tools	官方	★★★★★	动态暴露 Tool	Tool Context Engineering
Plan Mode	官方	★★★★☆	只读规划 -> 执行	State -> Capability Set
Handoff	官方	★★★☆☆	重启/迁移上下文	Context Reset / Transfer
pi-lens	社区	★★★★★	编辑后 lint/LSP/typecheck	Action -> Feedback -> Repair
pi-workflows	社区	★★★★★	多阶段工作流	State Machine + Tool Gating
toolwatch	社区	★★★★★	Tool 审计/审批	Runtime Governance Layer
pi-til-done	社区	★★★★☆	任务未完成则继续	Ralph Loop + Stop Condition
pi-worktrees	社区	★★★★☆	并行 Git workspace	Workspace Isolation
pi-processes	社区	★★★★☆	长期进程管理	Long-running Process Runtime
apply-patch-tool	社区	★★★★☆	结构化 patch 编辑	Structured Editing
skill-picker	社区	★★★★☆	显式 Skill 选择	Progressive Context + Human Override

3. 官方 Extension：最值得源码级拆解的 7 类

3.1 Sandbox：Execution Backend 与安全边界

Sandbox 是最值得优先阅读的官方示例之一。它不是简单在 bash 前面加一个字符串过滤器，而是把 “Tool Interface” 和 “Execution Backend” 解耦。



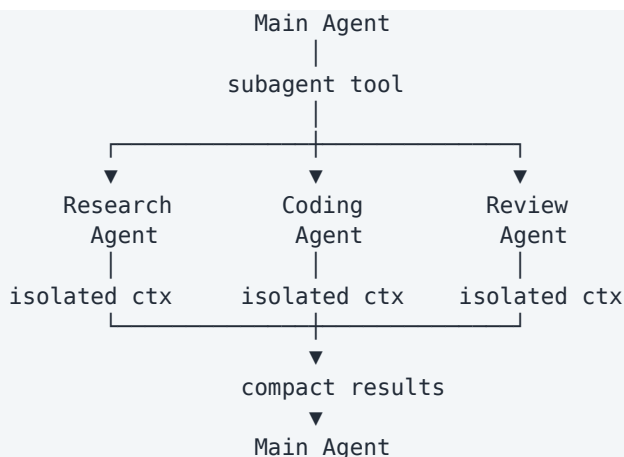
最关键的设计原则：模型只理解稳定 Tool Contract；Runtime 决定命令最终在 Local、Sandbox、Docker、SSH 还是 VM 中执行。

适合研究的问题

如何实现 filesystem allowlist、network deny-by-default、环境变量隔离、进程生命周期管理，以及 Tool Override 后如何保持模型接口不变。

3.2 Subagent：真正的价值是 Context Isolation

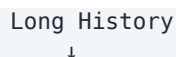
Subagent 常被误解为“多几个模型一起干活”。Pi 官方示例更重要的价值，是让不同 Agent 拥有独立 Context，并通过 Single / Parallel / Chain 等方式返回压缩后的结果。



Know-why：如果 research、logs、tests、implementation、review 全在 Main Agent 中，Context 会快速污染。Subagent 更像 Context Firewall。

3.3 Custom Compaction：从“聊天摘要”升级到 Agent Memory

长任务的难点不是把全部历史都保留，而是决定“未来推理真正需要什么”。Custom Compaction 允许把普通 summary 改造成结构化 working memory。



```

Structured Compactor
├─ Goal
├─ Current State
├─ Decisions
├─ Files Changed
├─ Failed Attempts
├─ Tests
├─ Constraints
├─ Next Actions
└─
↓
Working Memory

```

这一层特别适合研究 Agent Memory、Context Compression、长期任务恢复以及跨 Session 的状态沉淀。

3.4 Dynamic Tools : Tool Registry ≠ Active Tool Context

大型 Agent 很容易有几十到上百个 Tool。如果所有 Tool schema 永远进入 Context，会浪费 token，并提高模型选错 Tool 的概率。Dynamic Tools 的核心是把“注册”与“暴露给模型”分开。

```

100 registered tools
    ↓
task / capability router
    ↓
5 active tools
    ↓
LLM

```

这个思想可进一步演化成 Deferred Tool Loading：先暴露搜索/发现 Tool，需要时才激活真正的执行 Tool。

3.5 Plan Mode : 不是 Prompt，而是 Capability Gating

好的 Plan Mode 不是仅仅告诉模型“先计划”，而是改变当前 Agent State 对应的 Capability Set。

```

PLAN STATE
read    ✓
grep    ✓
write   ✗
edit    ✗

      ↓ approve / switch

EXECUTE STATE
read    ✓
edit    ✓
write   ✓
bash    ✓

```

这比“System Prompt 写一句不要修改文件”可靠得多，因为能力约束发生在 Runtime 层。

3.6 Tool Override : 稳定接口，替换执行实现

Tool Override 可以替换 read、bash、edit、write 的底层实现，用于加审计、路径保护、远程执行或沙箱。核心架构是 Interface / Backend 解耦。

3.7 Handoff : 主动重建 Context

Handoff 的核心不是简单“开新会话”，而是把当前任务的高价值状态压缩后交给新的 Session，从而主动清理历史噪声。对于超长 Coding 任务，这是一种显式 Context Reset。

4. 社区 Extension：适合日常 Coding Agent 的高价值组件

4.1 pi-lens：把 Coding Agent 变成闭环系统

pi-lens 的价值在于：模型完成 write/edit/bash 后，不等待人类发现错误，而是立即从 LSP、linter、type checker 等环境信号得到反馈。

```

Agent
↓
edit / write
↓
pi-lens
├─ LSP diagnostics
├─ lint
├─ typecheck
└─ language-specific checks
↓
structured diagnostics
↓
Agent repair
  
```

Know-how

Coding Agent 的可靠性来自“行动后的环境反馈”，而不是模型在行动前想得更多。最重要的循环是 Action -> Feedback -> Repair。

4.2 pi-workflows：Agent Workflow = State Machine + Tool Gating

pi-workflows 适合把 Research -> Plan -> Implement -> Test -> Review 变成明确 phase。每个 phase 可以拥有不同 Tool 权限和不同成功条件。

Research	->	Plan	->	Implement	->	Test	->	Review
read		read		edit		bash		read
grep		grep		write		tests		test
search		no-write		bash		no-deploy		review

与单纯 Prompt Workflow 相比，这种设计的优势是状态、权限、继续条件都属于 Runtime。

4.3 toolwatch：Runtime Governance Layer

toolwatch 适合学习 Tool 调用审计、敏感操作审批和长期审计日志。它对应生产 Agent 中非常重要的治理层：谁调用了什么 Tool、参数是什么、是否被阻止、是否需要人工批准。

4.4 pi-til-done：Ralph Loop / 自动继续

pi-til-done 体现一个简单但强大的思想：不要以“模型结束本轮”为完成条件，而是以“任务状态是否完成”为完成条件。

```

while not task_complete():
    run_agent()

stop when:
- task complete
- max iterations
- no progress
- repeated failure
- human abort
  
```

这非常适合和 Ralph Loop、长期 Coding Agent、自动修复任务一起比较研究。

4.5 pi-worktrees : 多 Agent 的 Workspace Isolation

当多个 Agent 并行修改同一代码库时，真正的冲突往往不是 Context，而是 Git branch、uncommitted changes 和文件系统。Worktree 是一个自然的隔离边界。

```
Repository
├─ worktree-A -> Agent A
├─ worktree-B -> Agent B
└─ worktree-C -> Agent C
```

4.6 pi-processes : 长期进程不能只靠 bash &

dev server、watcher、backend 等长期进程应该进入 Process Manager，而不是通过 shell 后台符号“甩出去”。成熟 Runtime 应保存 PID、状态、日志、restart/stop 生命周期。

4.7 apply-patch-tool : Structured Editing

结构化 patch 比整文件 rewrite 更容易控制修改范围，也更适合 review、diff 和失败恢复。对于 Coding Agent，编辑 Tool 本身就是一个值得单独优化的 Agent-Computer Interface。

4.8 skill-picker : 自动路由之外保留 Human Override

Skill 自动选择很方便，但概率路由不可避免会误判。skill-picker 的启发是：Automatic Routing + Explicit Human Override 往往比单一自动化更可靠。

5. 五个最值得深入研究的 Extension

如果目标不是“装更多功能”，而是理解现代 Agent Runtime，我建议优先拆这五类：

排名 / Extension	核心问题	关键能力	最值得沉淀的 Know-why
1. Sandbox	安全执行	Sandbox	Tool interface 与 execution backend 解耦；能力边界不依赖模型自觉。
2. Subagent	上下文隔离	Subagent	把多 Agent 重新理解为 context partition + result compression。
3. Custom Compaction	长期记忆	Custom Compaction	决定“未来推理需要什么”，而不是机械保留聊天历史。
4. Dynamic Tools	工具上下文	Dynamic Tools	Tool registry 与 active context 分离，控制 token 与 tool-selection entropy。
5. pi-workflows	流程编排	pi-workflows	把 Prompt Workflow 变成 Runtime State Machine + Capability Gating。

一个统一视角

Sandbox 解决“在哪里执行”；Subagent 解决“谁在什么上下文里做”；Compaction 解决“保留什么”；Dynamic Tools 解决“给模型什么能力”；Workflow 解决“什么时候允许什么能力”。这五个问题几乎覆盖 Agent Runtime 的主骨架。

6. 推荐组合：日常开发 / 高级 Agent / Runtime 研究

6.1 日常 Coding Agent

```
Pi
├─ pi-lens
├─ apply-patch-tool
├─ pi-processes
├─ toolwatch
└─ skill-picker
```

目标：提高修改后的反馈质量、减少危险 Tool 调用、改善结构化编辑，并把长期 dev process 变成可管理资源。

6.2 高级 Autonomous Agent

```
Pi
├─ Sandbox
├─ Subagent
├─ Custom Compaction
├─ Dynamic Tools
├─ pi-workflows
├─ pi-til-done
└─ pi-worktrees
```

目标：长期运行、可恢复、多 Agent 并发、严格能力边界和受控自动继续。

6.3 Agent Runtime 研究环境

```
Minimal Pi Core
+ Event / Hook layer
+ Dynamic Tool registry
+ Session Tree
+ Custom Compaction
+ Sandbox backend
+ Subagent executor
+ Workflow state machine
+ Tool audit / eval
```

目标：不是“使用某个产品”，而是把 Agent Loop、Context、Memory、Tool、Execution、Policy、Orchestration 这些机制逐层拆出来。

7. 源码阅读顺序与学习路径

1. **permission-gate** 理解最简单的 tool_call hook：为什么 Policy 应在 Runtime 而不是 Prompt。
2. **tool-override** 理解 Tool Interface 与底层执行实现如何解耦。
3. **dynamic-tools** 理解 Registered Tools 与 Active Tools 分离。
4. **sandbox** 把 Tool Override 扩展成真正的 execution boundary。
5. **custom-compaction** 进入 Context / Memory Runtime。
6. **plan-mode** 理解 Agent State 与 Capability Set。
7. **subagent** 理解 context isolation + process orchestration。
8. **pi-workflows** 理解多阶段 state machine 与 tool gating。
9. **pi-til-done** 理解 stop condition、progress detection 与长期循环。

```
Hook
↓
Middleware
↓
Tool Runtime
↓
Execution Runtime
↓
Context Runtime
↓
State Machine
↓
Multi-Agent
↓
Long-running Agent
```

8. 从 Extension 反推 Agent 架构原则

8.1 Minimal Core

Agent Loop 越薄越好。把 workflow feature 放在 Extension，不要在核心循环里不断加 if permission / if memory / if sandbox。

8.2 Policy ≠ Prompt

安全策略必须在 Runtime 执行，而不是依赖 System Prompt 要求模型“不要做危险操作”。

8.3 Tool Schema 是 Context

Tool schema 也是 Context token，必须和历史消息、RAG、Skill 一样被管理。

8.4 Subagent 的第一价值是 Context Partition

多 Agent 的收益来自任务和上下文的分割，而不是仅仅增加并发。

8.5 Memory 是选择，不是堆积

好的 Memory 系统核心是保留“决策、约束、状态、失败经验、下一步”，而不是无限扩展历史。

8.6 Workflow 应绑定 Capability

Research/Plan/Execute/Review 不只是标签；每个状态应绑定不同工具、权限和成功条件。

8.7 Environment Feedback 比更长思考更重要

Coding Agent 的闭环应主动获取 tests、LSP、lint、runtime logs，而不是只让模型增加推理。

8.8 Execution Backend 应可替换

read/bash/edit/write 应成为稳定 Agent-Computer Interface，底层可以自由切换 Local、Sandbox、SSH、VM。

9. 安装与安全注意事项

重要

Pi Extension 本质上是本地代码，通常拥有与你当前 Pi 进程相同的系统权限。安装社区 Extension 前应像审查普通开发依赖一样审查源码与发布来源。

- 优先阅读 Extension 是否覆盖 bash/read/edit/write，覆盖内置 Tool 意味着它可以改变真实执行行为。
- 查看是否读取环境变量、SSH key、云凭据、Git credential、浏览器 cookie 或项目 secret。
- 查看是否存在网络请求、自动上传、telemetry 或远程命令执行逻辑。
- 涉及长期自动循环时必须有 max iterations、no-progress detection、human abort。
- 涉及 Subagent / Worktree 时，应明确每个 Agent 的 workspace、branch 与可写范围。
- 涉及 Sandbox 时，应验证“默认拒绝”还是“默认允许”，以及网络和文件系统策略实际覆盖范围。

附录 A：Extension 选型速查表

目标	优先看	为什么
我想提高代码质量闭环	pi-lens	编辑后自动 diagnostics / lint / typecheck
我想限制危险 Tool	permission-gate / toolwatch	Runtime Policy + approval
我想让 Agent 运行在隔离环境	Sandbox / Gondolin	Execution isolation
我想并行研究/编码/Review	Subagent	Context isolation
我想管理上百个 Tool	Dynamic Tools	Tool context routing
我想跑很长的任务	Custom Compaction	Structured working memory
我想实现 Plan -> Execute	Plan Mode / pi-workflows	State + capability gating
我想自动循环直到完成	pi-til-done	Ralph loop + stop conditions
我想并行改多个分支	pi-worktrees	Git workspace isolation
我想管理 dev server	pi-processes	Long-running process manager
我想降低整文件重写风险	apply-patch-tool	Structured patch editing
我想让用户手工选 Skill	skill-picker	Human override for routing

附录 B：参考资料

- Pi 官方 Extensions 文档：pi.dev/docs/latest/extensions
- Pi 官方 Extension Examples：badlogic/pi-mono 或 earendil-works/pi 仓库中的 packages/coding-agent/examples/extensions
- Pi Sandbox / Subagent / Custom Compaction / Dynamic Tools / Plan Mode 等官方示例源码
- harms-haus/pi-extensions：pi-lens、pi-workflows、pi-til-done、pi-worktrees、pi-processes 等社区 Extension
- kcosr/pi-extensions：toolwatch、apply-patch-tool、skill-picker 等社区 Extension

说明：本报告依据本次对话前序检索与整理结果编写，重点是 Extension 的架构价值与 Agent Runtime know-how，而不是作为“插件商店完整目录”。社区项目版本和维护状态可能变化，实际安装前应再次检查仓库最新状态。

推荐的下一步

如果要继续深入，最有价值的不是再收集 Extension 名单，而是挑 Sandbox、Subagent、Custom Compaction、Dynamic Tools、pi-workflows 五个项目做源码级执行轨迹：入口 -> 注册 Hook -> 修改 Runtime -> Tool/Context/Session 变化 -> 回到 Agent Loop。